Aarav Sharma

+91 98765 43210 | aarav.sharma@example.com | LinkedIn | GitHub | LeetCode

Summary

Computer Science undergraduate focused on backend engineering, distributed systems and cloud-native development. Experienced in building REST and gRPC microservices in Go and Python, designing normalised PostgreSQL schemas, and shipping containerised services with Docker, Kubernetes and GitHub Actions CI/CD. Hands-on with Redis caching, Apache Kafka event streaming, consensus-based replication and observability using Prometheus, Grafana and OpenTelemetry. Strong foundation in data structures, algorithms, operating systems and system design, with a habit of writing tested, well-documented code and owning services from design through deployment.

Technical Skills

Languages: Go, Python, Java, SQL, C++
Backend: REST APIs, gRPC, FastAPI, Gin, Spring Boot, Microservices
Databases & Caching: PostgreSQL, Redis, MongoDB
Messaging: Apache Kafka, RabbitMQ
Cloud & DevOps: Docker, Kubernetes, AWS (EC2, S3, RDS), GitHub Actions, CI/CD, Linux
Observability & Testing: Prometheus, Grafana, OpenTelemetry, Jaeger, pytest, Go testing, k6
Core CS: Data Structures & Algorithms, System Design, Operating Systems, DBMS, Computer Networks

Projects

Distributed Rate Limiter – Token-bucket API gateway GitHub

- Built a distributed rate-limiting gateway in Go using Redis Lua scripts for atomic token-bucket updates across multiple instances.
- Supported token-bucket, fixed-window and sliding-window algorithms, selectable per route through YAML configuration.
- Exposed per-client limits through a REST admin API and gRPC middleware, with configuration stored in PostgreSQL.
- Containerised with Docker and deployed on Kubernetes, with Prometheus metrics and Grafana dashboards for throttled requests.
- Added load tests with k6 and integration tests in GitHub Actions CI on every pull request.

Event-Driven Order Service – Kafka-based microservices *Python, FastAPI, Kafka, PostgreSQL*

- Designed order, payment and inventory microservices in FastAPI communicating through Apache Kafka events.
- Implemented the transactional outbox pattern and idempotent consumers to guarantee exactly-once order processing.
- Traced requests end to end with OpenTelemetry and Jaeger, and documented all APIs with OpenAPI.
- Handled failures with retries, dead-letter topics and saga-based compensation to keep orders consistent across services.
- Ran the full stack locally with Docker Compose and tested event flows with pytest and Testcontainers in CI.

Key-Value Store – Raft-replicated storage engine *Go, gRPC, Raft*

- Implemented a replicated key-value store in Go using the Raft consensus algorithm for leader election and log replication.
- Built a gRPC client API with linearizable reads, snapshotting and log compaction.
- Verified fault tolerance with automated tests that kill and restart nodes during writes.
- Persisted the replicated log with a write-ahead log and crash recovery, and exposed cluster health metrics to Prometheus.

Education

B.Tech, Computer Science and Engineering 2022 – 2026
Example Institute of Technology CGPA: 8.70/10

Class XII (CBSE) 2022
Example Public School 92.4%

Class X (CBSE) 2020
Example Public School 94.0%

Certifications

- AWS Certified Cloud Practitioner – Amazon Web Services
- Introduction to Kubernetes (LFS158) – The Linux Foundation
- Apache Kafka Fundamentals – Confluent

Achievements

- Finalist, college-level hackathon (backend track), among 120 teams.
- Solved 500+ problems on LeetCode; Knight badge.
- Technical lead, college coding club: ran weekly backend and system design sessions for juniors.